SOFTWARE ARCHITECTURE DOCUMENT

Traction Inverter

MCUs:	STM32H723ZG + STM32G474RCTx
Operating Temp:	−40°C to +85°C
Prepared by:	Thomas Liao
Reviewed by:	(not yet reviewed)
Date:	July 8, 2026
Version:	1.4

University of California, Santa Cruz — Corzine Lab

Document History

Date	Version	Author	Description of Changes
June 13, 2026	1.0	Project Team	Initial release.
June 14, 2026	1.1	Project Team	TIM1 not HRTIM, multi-modulation, 50kHz oversampling, corrected ADC, 1oo3 temp, RTE interface.
June 14, 2026	1.2	Project Team	Major fixes: default 2kHz SVPWM (was 6kHz), min 300Hz (was 1kHz), ADC 0-3.3V scaling (not 1.67V), removed all GPIO pin assignments (refer to CubeMX), added SPI FRAM storage with write protection, input validation framework, real-time loss estimator + thermal model, power rail switching (CAN/sensor/gate driver kills), firmware update via UART/USB or CAN.

Table of Contents

1. Introduction and Scope

1.1 Purpose

This Software Architecture Document (SWAD) defines the software architecture for the STM32H723ZG-based traction inverter / Vehicle Control Unit (VCU) used in open-source aftermarket electric motorcycle conversion platforms. It implements the functional safety requirements specified in the companion Hazard Analysis and Risk Assessment (HARA, Version 3.0) and the cybersecurity requirements specified in the companion Threat Analysis and Risk Assessment (TARA, Version 1.0).

The architecture targets ISO 26262 ASIL-C safety integrity for motor control functions while remaining fully open-source, auditable, and free of vendor lock-in or OTP security fuses. All design decisions are traceable to specific Functional Safety Requirements (FSRs) or Cybersecurity Requirements (CSRs).

1.2 Scope

This document covers the complete software architecture:

- Four-layer software architecture (HAL/BSP, Safety, Control, Application)
- Six-state system state machine with full transition logic
- Field-Oriented Control (FOC) with multiple modulation schemes (SPWM, SVPWM, SHEPWM, N-Pulse, RSVM, RCFM)
- Multi-rate oversampled current sensing (50 kHz bandwidth, 16-bit primary + 12-bit redundant)
- Real-time power loss estimator with thermal model for die temperature prediction
- Power rail switching — controlled shutdown of CAN, sensor, and gate driver supplies
- SPI FRAM for high-endurance non-volatile parameter storage with write protection
- Input validation framework for all user-configurable parameters
- All safety-critical modules with FSR traceability
- CAN protocol reference (user-tailorable)
- Flash and RAM memory layouts
- Bootloader with firmware update via UART/USB or CAN
- HMAC-SHA256 firmware signing (open-source trust model)
- RTE (Real Time Examiner) open-source configuration tool interface

This document does *not* cover:

- Hardware circuit design or PCB layout (see HW design files)
- Mechanical packaging or thermal design
- Detailed threat analysis (see TARA) or hazard analysis (see HARA)

- GPIO pin assignments (generated by STM32CubeMX, see project .ioc file)
- Implementation schedule (managed externally)
- **Traction motor design or construction** — the motor is an external product supplied by the vehicle manufacturer or aftermarket vendor
- **Encoder or resolver design** — the rotor position sensor is part of the external motor; this document only covers the interface to it
- **BMS, IO board, charger, or display design** — these are external CAN nodes; this document covers the CAN interface only

> **Two Evaluations: With and Without Safety Coprocessor**
>
> This document presents two safety integrity evaluations for the same hardware: (1) **Current design (no coprocessor):** ASIL-C achievable for most safety goals, ASIL-A for H-06/H-16/H-17. This is the hardware that exists today. (2) **Future design (with coprocessor):** ASIL-D achievable for SG-01 and SG-13 via ASIL decomposition. The coprocessor is a future enhancement, not current hardware. Where ASIL ratings differ between these two configurations, both are stated explicitly.

1.3 Design Philosophy: User-Configurable, Safe, Reliable

This system is designed from the ground up to be **user-configurable, safe, and reliable**. Every aspect of the control strategy that does not compromise safety is exposed to the user through the RTE (Real Time Examiner) open-source tool:

- **Modulation scheme:** SPWM, SVPWM, SHEPWM, N-Pulse (with variants), RSVM, RCFM — selectable at runtime
- **Switching frequency:** 300 Hz minimum, user-adjustable up to 16 kHz (default 2 kHz SVPWM)
- **FOC status:** Preliminary testing completed on Zero 75-10 motor (bench, open-loop V/Hz and closed-loop current control). Full torque control and dyno validation pending.
- **Torque maps:** Four user-defined 16-point curves, plus Sport and Eco presets
- **Safety thresholds:** DC-link OV/UV, temperature limits, throttle plausibility delta — all adjustable within hard bounds
- **PI gains:** Id and Iq controller gains, adjustable for different motors
- **CAN protocol:** Frame IDs, scaling factors, periods — all user-tailorable
- **Custom parameters:** 32 float + 32 uint32_t user-defined slots for application-specific tuning

However, **safety-critical parameters have hard-coded absolute bounds** that cannot be overridden. For example, the IGBT temperature hard cap of 100C cannot be raised above 100C even if the user requests it. The DC-link overvoltage threshold cannot be set above the hardware rating. These bounds are compiled into the firmware and are not modifiable via RTE.

Parameter storage uses an external SPI FRAM chip (not internal flash) for unlimited write endurance. A hardware write-protection pin prevents accidental parameter corruption during firmware updates or fault conditions.

1.4 Target Hardware

Target Hardware Summary

Parameter	Value	Relevance
MCU	STM32H723ZG, Cortex-M7, 550 MHz	Single-core, FPU, cache, dual-bank flash
Flash	1 MB dual-bank (2 x 512 KB)	Bootloader + application + swap for OTA
RAM	564 KB (AXISRAM + SRAM1-4)	FOC buffers, stack, ECC on AXISRAM
PWM + FOC	TIM1 Advanced Timer (6 comp. outputs)	PWM carrier 300Hz-16kHz; FOC loop runs at same frequency; hw dead-time, break input
Gate Driver	NCV57001 x 6 (AEC-Q100)	DESAT, UVLO, active Miller clamp
Current Sensors	Tamura LA37S x 4 (3-phase + DC link)	+/-1200 A, 1.042 mV/A, 0-3.3V to ADC
Temp Sensors	NTC thermistor x 1 per IGBT module (3 modules, 3 sensors total)	1-out-of-3 voting, highest wins
CAN	Dual FDCAN, up to 8 Mbps	BMS, IO board, charger, diagnostic
FRAM	SPI-connected FRAM (e.g., Cypress FM25W256)	Unlimited endurance param storage, WP pin
Encoder	Sin/Cos or resolver (configurable)	Rotor position for FOC
DC-Link	102 V - 350 V (nominal range)	Overvoltage/undervoltage monitoring
Motor Power	Up to 100 kW peak	Peak current ~280 A (at 350V)

2. References and Standards

Normative References

Reference	Title	Usage
ISO 26262-1:2018	Functional safety -- Vocabulary	Definitions and terminology
ISO 26262-3:2018	Part 3: Concept phase (HARA)	Hazard analysis, ASIL assignment
ISO 26262-6:2018	Part 6: Product development -- software	Software architectural design
ISO/SAE 21434:2021	Cybersecurity engineering	Cybersecurity requirements (TARA)
STM32H723xx RM	Reference Manual (RM0468)	Register-level programming
NCV57001 DS	NCV57001 Datasheet	Gate driver timing and protection
Tamura LA37S	LA37S Current Transducer Datasheet	Current sensing specification
FM25W256	Cypress 256-Kbit SPI FRAM	Non-volatile parameter storage

3. Software Architecture Overview

3.1 Design Principles

1. **User-configurable, safety-bounded:** Every non-safety-critical parameter is adjustable via RTE. Safety-critical parameters have hard-coded min/max bounds that cannot be overridden.
2. **Deterministic timing:** No dynamic memory allocation in safety paths. No RTOS. Superloop plus interrupts only.
3. **Hardware fault containment:** Gate driver faults trigger TIM1 break input for sub-microsecond hardware SSO, independent of software.
4. **Multi-path safe state:** Three complementary paths to SSO: (1) TIM1 break input (hardware, <100 ns, independent of MCU CPU), (2) MCU GPIO → PMIC EN gate driver supply kill (active, ~10 us, MCU-dependent, basic FLT feedback via NCV57001 VDD_UVLO), (3) loss of shared 3.3V rail → NCV57001 internal pull-down (passive, automatic, no software).
5. **Fail-safe defaults:** SSO is the hardware default. Any unrecognized condition results in SSO.
6. **Open-source trust model:** No OTP fuses, no vendor-locked keys. HMAC-SHA256 with user-managed keys. Physical access is acceptable risk.
7. **Multi-modulation:** SPWM, SVPWM, SHEPWM, N-Pulse family, RSVM, RCFM — selectable at runtime via RTE.
8. **Current loop at switching frequency:** FOC runs at the PWM carrier frequency (300 Hz - 16 kHz), not at a fixed rate. The current loop rate IS the switching frequency. ADC oversamples at an integer multiple (n×) of the PWM frequency, phase-locked to avoid beat noise.
9. **High-bandwidth sensing:** ADC oversampling up to 48 kSPS provides ~24 kHz effective current sense bandwidth, sufficient for low-inductance motors at low switching frequencies.
10. **Layered abstraction:** Safety mechanisms isolated in a dedicated layer, independent of control algorithms.
11. **Defensive programming:** All inputs range-checked. All state transitions validated. Full input validation framework (Section 7.7).

3.2 Layer Diagram

Software Layer Summary

Layer	Modules	Safety Integrity	Timing
Application	Throttle, CAN Comms, DCLink Mon, Temp Mon (1oo3), POST, RTE, Input Validator	ASIL-B	1 ms - 100 ms

Control	FOC, Modulation Library, PI Controllers, State Machine, Torque Commander, Loss Estimator	ASIL-C	PWM period (3.3 ms @ 300 Hz - 62.5 us @ 16 kHz)
Safety	Fault Manager, Safe State Manager, Watchdog Manager, PSU Monitor, Rail Switch	ASIL-C	1 ms - 100 ms
HAL/BSP	TIM1, ADC, FDCAN, SPI (FRAM), GPIO, EXTI, Flash, CRC	ASIL-B	Hardware-driven

4. Hardware Abstraction Layer (HAL/BSP)

The HAL/BSP layer provides register-level drivers for all STM32H723ZG peripherals. This is the only code that directly accesses hardware registers. All upper layers call HAL functions.

> **GPIO Pinout — Refer to STM32CubeMX**
>
> All GPIO pin assignments, alternate function mappings, and peripheral pinouts are generated from the STM32CubeMX project configuration file (`.ioc`). The generated source files (`main.c`, `gpio.c`, `tim.c`, `adc.c`, `fdcan.c`, `spi.c`) are checked into version control and must not be hand-edited. Re-generate via CubeMX when changes are needed. This document does not list specific GPIO pins.

4.1 TIM1 PWM Driver

TIM1 (Advanced Control Timer) serves **dual roles**: (1) it generates six complementary PWM outputs at the user-selected carrier frequency with hardware dead-time and break input; (2) its update interrupt triggers the FOC current loop at the same frequency. The ADC is also triggered by TIM1 at an integer multiple (n×) of the PWM frequency for oversampled current sensing.

TIM1 Timing Parameters

Parameter	Default	Range	Notes
TIM1 clock	275 MHz	Fixed	APB2 timer clock
Default PWM frequency	2 kHz	300 Hz - 16 kHz	User-configurable via RTE; SVPWM default
Minimum PWM frequency	300 Hz	Hard limit	Below this: audible noise, excessive current ripple
Maximum PWM frequency	16 kHz	Hard limit	Above this: excessive switching losses
Dead time	2.0 us	500 ns - 4 us	User-configurable; prevents shoot-through
Break input	TIM1_BKIN	Active low	OR'd gate driver fault aggregation
Break response	SSO (MOE cleared)	Hardware	<100 ns, zero software latency
Re-arm	Software only	Required	Prevents auto-restart after fault
ADC trigger	TIM1_TRGO	n× PWM freq (up to 48 kSPS)	Phase-locked oversampling, sinc3 decimation

FOC trigger	TIM1 update (UIF)	PWM switching frequency	Current loop rate = carrier frequency

4.1.1 TIM1 Break Input — Hardware SSO

When any NCV57001 gate driver detects DESAT or UVLO, its FAULT output (active low) asserts. Six FAULT outputs are OR'd together and fed to TIM1_BKIN. TIM1 hardware immediately clears MOE (main output enable), forcing all six PWM outputs to the inactive (SSO) state within <100 ns. A break interrupt then fires to log the fault.

4.2 ADC Driver — Multi-Rate Oversampled

The ADC subsystem achieves 50 kHz effective current sense bandwidth through multi-rate oversampling. Five samples are taken per PWM period, decimated via sinc3 filtering. This is essential for low-inductance motors running at low switching frequencies (300 Hz - 2 kHz) where current ripple would otherwise alias into the control loop.

ADC Channel Assignment

Signal	ADC1 (16-bit)	ADC2 (12-bit)	ADC3 (12-bit)
I_U (phase U current)	INx (16-bit)	—	—
I_V (phase V current)	INx (16-bit)	INx (12-bit)	—
I_W (phase W current)	INx (16-bit)	INx (12-bit)	INx (12-bit)
REF_U (I_U reference)	INx (16-bit)	—	—
REF_V (I_V reference)	INx (16-bit)	INx (12-bit)	—
REF_W (I_W reference)	INx (16-bit)	INx (12-bit)	INx (12-bit)
V_DC (DC link voltage)	INx (16-bit)	—	—
Throttle main	INx (16-bit)	—	—
Throttle check	INx (16-bit)	—	—
T_IGBT[0-2]	INx (12-bit)	—	—
VREFINT (3.3V rail)	Internal	—	—

ADC1 at 16-bit is the primary measurement path for all control-critical signals. ADC2 and ADC3 at 12-bit provide lower-resolution redundant channels for cross-check. The exact ADC input channel numbers (INx) are assigned via STM32CubeMX and vary with board revision.

> **ADC Resolution: 16-bit Primary + 12-bit Redundant**
>
> The 12-bit redundant channels (ADC2, ADC3) have approximately 5.3x coarser resolution than the 16-bit primary (ADC1). For the purpose of fault *detection* — the safety-relevant function — this difference is not operationally significant. A 12-bit channel can reliably detect catastrophic sensor faults: broken wire (reading at rail), stuck output (constant value regardless of current), and large gain errors (>20% deviation). These are the failure modes that matter for safety. Subtle gain errors or small calibration drift are not safety-critical because the 16-bit primary channel remains the control input. The 12-bit channels serve as a "sanity check" on the primary, not as a precision backup. A second 16-bit ADC would provide higher-fidelity redundancy but is not required for ASIL-C fault detection.

4.2.1 Oversampling Configuration

At 2 kHz PWM (default), 5 ADC samples per period = 10 kSPS per channel. The sinc3 decimator produces one filtered output every 5 raw samples, giving an effective 2 kHz control update rate with 25x noise reduction. The raw 10 kSPS stream is also analyzed for peak-to-peak ripple amplitude — excessive ripple flags a sensor or connection fault.

4.3 Current Sensor Validation

Each Tamura LA37S outputs a measurement signal and an independent reference signal. Both are conditioned to 0-3.3V for the 16-bit ADC.

Tamura LA37S Electrical Characteristics

Parameter	Value	Note
Measurement range	-1200 A to +1200 A	Bidirectional
Sensitivity	1.042 mV/A	At sensor output pins
Center voltage (0 A)	2.5 V	Internal reference output
Input to ADC	0 - 3.3 V	After signal conditioning
Resolution (16-bit, 3.3V ref)	~36.6 mA/LSB	2400 A span / 65536

4.3.1 VREF Validation

The reference channel for each sensor is checked on every conversion. After signal conditioning (scaling to 3.3V range), the reference must be within:

VREF Acceptance Window (After Conditioning to 3.3V)

Min	Typical	Max

VREF (after conditioning)	1.63 V (2.48V scaled)	1.65 V (2.50V scaled)	1.67 V (2.52V scaled)

Deviation outside this window marks the sensor as degraded. FAULT_SENSOR_VREF is raised. The control loop continues using remaining healthy channels.

4.3.2 Zero-Point Check

During a controlled zero-current window, all phase sensors are read. Each must be within +/-20% of zero (i.e., +/-240 A equivalent). Gross deviations indicate a broken wire or stuck output. Results update the running zero-offset calibration.

4.3.3 DC-Link Back-Calculation

Independent sanity check of DC-link measurements:

$$V_{dc\_calc} = (3 \times V_{ph} \times I_{ph} \times \cos(\varphi)) / (I_{dc} \times \eta)$$

Where V_{ph} is AC phase voltage (RMS), I_{ph} is AC phase current (RMS), cos(φ) is motor power factor, η is inverter+motor efficiency (0.93-0.97, user-configurable), and I_{dc} is measured DC bus current. Runs at 100 Hz. Logs a warning if calculated V_{dc} deviates from measured V_{dc} by more than 15%.

4.4 FDCAN Driver

Dual FDCAN modules provide CAN communication. The protocol is intentionally user-tailorable — the frame definitions in Section 10 are a starting point. Users modify IDs, scaling, and layouts via RTE.

4.5 SPI FRAM — Non-Volatile Parameter Storage

An external SPI FRAM chip (e.g., Cypress FM25W256, 256-Kbit) stores all user-configurable parameters, runtime counters, and fault logs. FRAM provides unlimited write endurance (10^14 write cycles) compared to flash (~10,000 cycles), making it suitable for frequently updated data.

FRAM Storage Map

Address Range	Content	Update Frequency
0x0000 - 0x00FF	Parameter header (magic, version, CRC)	On parameter commit only
0x0100 - 0x04FF	Torque maps (4 x 16-point curves + metadata)	On user commit via RTE
0x0500 - 0x06FF	Safety thresholds (temp, DC-link, throttle)	On user commit via RTE
0x0700 - 0x07FF	CAN protocol config (IDs, scaling, periods)	On user commit via RTE
0x0800 - 0x08FF	Motor parameters (Ld, Lq, lambda_m, poles)	On user commit via RTE

0x0900 - 0x09FF	Modulation config (scheme, freq, dead time)	On user commit via RTE
0x0A00 - 0x0AFF	Custom user parameters (32 float + 32 uint32_t)	On user commit via RTE
0x0B00 - 0x0B3F	Odometer (total km, 64-bit)	Every 1 km
0x0B40 - 0x0B4F	Hour counter (motor run hours, 32-bit)	Every 1 hour
0x0B50 - 0x0B53	Boot counter (total resets, 32-bit)	Every boot
0x0B60 - 0x1F5F	Fault log ring buffer (256 entries x 20 bytes)	On every fault event
0x1F60 - 0x1FFF	Reserved	—

4.5.1 Write Protection

The FRAM chip has a hardware write-protect (WP) pin connected to a GPIO. WP is asserted (high) during normal operation, preventing accidental writes. WP is de-asserted only during:

1. Explicit parameter commit via RTE (user-initiated save)
2. Fault log write (atomic, WP toggled for single write cycle)
3. Odometer/hour counter update (periodic, WP toggled briefly)

> **FRAM WP Pin Failure Detection**
>
> If the WP pin GPIO fails (stuck low), FRAM is still protected because the SPI transaction must carry the correct write-enable (WREN) opcode. A stuck-low WP pin is detected during POST (test 13).

If the WP pin GPIO fails (stuck low), the FRAM is still protected from corruption because the SPI transaction must also carry the correct write-enable (WREN) opcode sequence. A failed WP pin is detected during POST.

4.6 Power Rail Switching

Three power rails can be switched off under software control. These complement the hardware SSO paths (TIM1_BKIN, shared 3.3V passive) but are not independent of the MCU:

Power Rail Control

Rail	Supply	Control	Use Case
Gate driver power	Murata MGJ2D121509MPC-R7 x 6	PMIC EN (shared enable)	Primary SSO path: kills all 6 gate drive supplies simultaneously, forcing IGBT gates to 0V via NCV57001 active pull-down

Current sensor power	+5V sensor supply	GPIO-controlled load switch	Disables current sensors for safe state or diagnostics
CAN transceiver power	12V to 5V DC-DC for ISO1042	GPIO-controlled load switch	Disables CAN bus for EMI reduction or fault isolation

4.6.1 Gate Driver Supply Kill — Primary SSO Path

The gate driver supply kill is the most aggressive safe-state action. When the PMIC EN line is pulled low, all six Murata isolated DC-DC converters shut down simultaneously. Within microseconds, the NCV57001 gate drivers detect the loss of their isolated supply and activate internal pull-downs, forcing all IGBT gates to 0V.

FLT Verification on Supply Kill: When the NCV57001 detects VDD_UVLO (loss of gate drive supply), it asserts its FLT output (active low). Since all six FLT outputs are OR'd together and fed to TIM1_BKIN, the MCU can verify that a PMIC EN kill actually worked by reading the FLT line after a short delay (~100 us). If FLT is asserted (low), the gate driver supply has been successfully shut off. If FLT remains de-asserted (high), the supply kill failed and the system must use TIM1_BKIN SSO as the fallback. This provides a basic feedback mechanism — not as robust as a dedicated voltage sense on each supply, but better than no feedback at all. Independent supply voltage monitoring (via ADC or isolated status signal) is a recommended future enhancement for higher diagnostic coverage.

```
/* RailSwitch.c — software-controlled power rail management */
void RailSwitch_KillGateDrivers(void) {
    GPIO_WritePin(PMIC_EN_GPIO_Port, PMIC_EN_Pin, GPIO_PIN_RESET);
    /* All 6 Murata supplies off within ~10 us */
    /* NCV57001 internal pull-downs force gates to 0V within ~1 us */
    rail_status.gate_driver_power = false;
    FaultMgr_SetFault(FAULT_RAIL_GATE_DRIVER_KILL, 0);
}

void RailSwitch_KillCANPower(void) {
    GPIO_WritePin(CAN_PWR_EN_GPIO_Port, CAN_PWR_EN_Pin, GPIO_PIN_RESET);
    /* ISO1042 transceivers powered off */
    rail_status.can_power = false;
}

void RailSwitch_KillSensorPower(void) {
    GPIO_WritePin(SENSOR_PWR_EN_GPIO_Port, SENSOR_PWR_EN_Pin, GPIO_PIN_RESET);
    /* Current sensor +5V supply off */
    rail_status.sensor_power = false;
}
```

5. Safety Layer

5.1 Fault Manager (FaultMgr)

Global fault registry — single source of truth for system health. The State Machine queries this on every tick.

Fault Classification

Fault Name	Class	Source	FSR	Response
FAULT_GATE_DRIVER	Critical	TIM1_BRK ISR	FSR-03,04	Immediate SSO + SAFE
FAULT_OVERCURRENT_HW	Critical	ADC comparator	FSR-05	Immediate SSO + SAFE
FAULT_OVERCURRENT_SW	Critical	FOC ISR	FSR-05	Immediate SSO + SAFE
FAULT_ADC_FAIL	Critical	ADC cross-check	FSR-06	Ramp down + SAFE
FAULT_SENSOR_VREF	Non-critical	Current sensor VREF	FSR-06	Log, use healthy ch
FAULT_SENSOR_ZERO	Non-critical	Zero-point check	FSR-06	Log, flag recal
FAULT_THROTTLE_PLAUS	Critical	Throttle	FSR-07	Zero torque + FAULT
FAULT_DC_LINK_OV	Critical	DCLink Mon	FSR-09	Immediate SSO + SAFE
FAULT_DC_LINK_UV	Critical	DCLink Mon	FSR-09	Zero torque + FAULT
FAULT_IGBT_OT	Critical	TempMonitor	FSR-11	Ramp down + FAULT
FAULT_IGBT_OT_WARN	Non-crit	TempMonitor	FSR-11	Log, limit 50%
FAULT_ENCODER	Critical	ENC_Monitor	FSR-15	Ramp down + FAULT
FAULT_CAN_TIMEOUT	Critical	CAN heartbeat	FSR-17	Ramp down + FAULT
FAULT_WATCHDOG	Critical	IWDG/WWDG	FSR-18	Reset then POST
FAULT_ECC_RAM	Critical	NMI handler	FSR-20	Log + SAFE
FAULT_POST_FAIL	Critical	POST	FSR-21	INIT then SAFE
FAULT_RAIL_GATE_DRIVER_KILL	Critical	RailSwitch	FSR-03	SSO via supply kill

5.2 Safe State Manager (SSM)

Three complementary paths to SSO:

1. **TIM1 break input (hardware, active):** Gate driver FAULT triggers TIM1_BKIN, MOE cleared, <100 ns. Independent of MCU CPU state.
2. **Gate driver supply kill (software + hardware, active):** MCU GPIO → PMIC EN low → Murata supplies off → NCV57001 UVLO pull-down, ~10 us. MCU-dependent. **Basic feedback:** NCV57001 asserts FLT on VDD_UVLO, which the MCU can read via TIM1_BKIN. Independent supply voltage monitoring is a recommended future enhancement.
3. **Shared 3.3V rail loss (hardware, passive):** Loss of 3.3V (shared by MCU and NCV57001 logic) → NCV57001 VDD lost → internal pull-down → SSO. No software intervention. Only effective for total power collapse.

Path 1 is the primary diagnosed path (TIM1_BKIN hardware). Path 2 complements it but depends on MCU GPIO functioning. Path 3 is a passive fallback for total power loss. Path 2 and Path 3 are related (both involve power) but operate under different conditions. A coprocessor with an independent switch would provide a fully independent third active path.

5.3 Watchdog Manager

Watchdog Configuration

Watchdog	Type	Timeout	Service	Action
IWDG	Independent (32 kHz RC)	100 ms	Every 50 ms	System reset
WWDG	Window (sys clock)	83.3 ms	Every 50 ms	System reset
External	TPS3823	1.6 s	Every 100 ms	System reset

5.4 Power Supply Monitor

Internal VREFINT channel monitors 3.3V regulator indirectly. Window: 3.0 V - 3.6 V. Outside this range: safe state entry.

6. Control Layer

6.1 Field-Oriented Control (FOC)

FOC runs in the **TIM1 update interrupt at the PWM switching frequency** (priority 0, cannot be preempted). The current loop rate IS the switching frequency: 300 Hz minimum, 16 kHz maximum, user-selectable via RTE. TIM1 generates the PWM outputs and triggers the FOC computation on every update event. Duty cycles are written to TIM1's preload (shadow) registers and applied at the next PWM period boundary — this is standard double-buffering.

6.1.1 Oversampled ADC Triggering

The ADC is triggered by TIM1 at an **integer multiple** (n×) of the PWM frequency, up to the hardware sampling rate limit. At each PWM period, n ADC samples are taken and decimated (boxcar or sinc3 filter) to produce one filtered current reading for the FOC loop:

ADC Oversample Ratio vs. Switching Frequency

PWM Frequency	Oversample Ratio (n)	ADC Sample Rate	Effective BW	Filter
16 kHz	3	48 kSPS	~24 kHz	Sinc3
8 kHz	6	48 kSPS	~24 kHz	Sinc3
4 kHz	12	48 kSPS	~24 kHz	Sinc3
2 kHz (default)	24	48 kSPS	~24 kHz	Sinc3
1 kHz	48	48 kSPS	~24 kHz	Sinc3
500 Hz	96	48 kSPS	~24 kHz	Sinc3
300 Hz	160	48 kSPS	~24 kHz	Sinc3

The integer multiple ensures ADC sampling is phase-locked to the PWM carrier, avoiding beat-frequency noise. The ADC sample rate is capped at ~48 kSPS per channel (hardware limit of 16-bit mode with conversion time). The sinc3 decimator provides ~60 dB of alias rejection. Raw oversampled data is also analyzed for peak-to-peak ripple amplitude — excessive ripple flags a sensor or connection fault.

6.1.2 FOC ISR Implementation

```
/* TIM1_UP_IRQHandler — FOC runs at PWM switching frequency */
void TIM1_UP_IRQHandler(void) {
    TIM1->SR &= ~TIM_SR_UIF;

    /* Read decimated currents (sinc3 output, one sample per PWM period) */
    float iu = adc_decimated[0];
    float iv = adc_decimated[1];
    float iw = adc_decimated[2];

    /* Read rotor angle */
    float theta_e = Encoder_GetElectricalAngle();

    /* Clarke transform */
    float i_alpha = iu;
    float i_beta  = (iu + 2.0f * iv) * ONE_OVER_SQRT3;

    /* Park transform */
    float cos_t = cosf(theta_e), sin_t = sinf(theta_e);
    float id =  i_alpha * cos_t + i_beta * sin_t;
    float iq = -i_alpha * sin_t + i_beta * cos_t;

    /* PI controllers */
    float iq_ref = TorqueCmd_GetIqRef();
    float vd = PI_Update(&pi_id, 0.0f - id);
    float vq = PI_Update(&pi_iq, iq_ref - iq);

    /* Voltage limit */
    float v_max = DCLink_GetVdc() * ONE_OVER_SQRT3 * MAX_MODULATION;
    float v_mag_sq = vd*vd + vq*vq;
    if (v_mag_sq > v_max*v_max) {
        float scale = v_max / sqrtf(v_mag_sq);
        vd *= scale; vq *= scale;
    }

    /* Inverse Park */
    float v_alpha = vd * cos_t - vq * sin_t;
    float v_beta  = vd * sin_t + vq * cos_t;

    /* Modulation dispatch */
    float du, dv, dw;
    mod_scheme_table[active_scheme](v_alpha, v_beta, DCLink_GetVdc(), &du, &dv, &dw);

    /* Write to TIM1 preload registers (applied next PWM period) */
    TIM1->CCR1 = (uint16_t)(du * TIM1->ARR);
    TIM1->CCR2 = (uint16_t)(dv * TIM1->ARR);
    TIM1->CCR3 = (uint16_t)(dw * TIM1->ARR);
}
```

6.1.3 Trigonometric Functions: CORDIC vs. Software

6.1.2 Trigonometric Functions: CORDIC vs. Software

The FOC ISR uses `cosf()`, `sinf()`, and `sqrtf()` from the CMSIS-DSP library. The STM32H723 includes a **CORDIC hardware accelerator** that computes sin, cos, and sqrt in ~10-20 cycles each with deterministic execution time:

- CORDIC: ~20-40 ns per operation (hardware, cache-independent)
- Software: ~150-200 ns per operation (library, cache-dependent)

CORDIC is recommended for production. Software trig is used during bench development for portability. Migration to CORDIC planned before vehicle testing. The 5 us WCET claim for SHEPWM worst-case must be validated with actual measurements on production hardware.

6.2 Modulation Scheme Library

Supported Modulation Schemes

Scheme	Type	Bus Util	Use Case
SPWM	Sinusoidal	78.5%	Baseline, simple
SVPWM	Space Vector	90.7%	Default (2 kHz), best utilization
SHEPWM	Harmonic Elimination	Variable	Low noise, specific freq
N-Pulse	Synchronous	High	Min switching loss, low speed
N-Pulse Wide	Synchronous	High	Wide pulse, low speed high torque
Custom	Synchronous	Variable	User-defined via RTE
RSVM	Randomised	90.7%	EMI spreading
RCFM	Randomised	90.7%	Acoustic noise spreading

6.2.1 Modulation Scheme Selection and Transitions

The active modulation scheme is selected via the RTE configuration tool and stored as an enum in FRAM. The scheme can be changed at runtime through RTE, but only under safe transition conditions.

Transition Conditions

A modulation scheme transition is permitted only when:

- **Torque reference is zero** (no-load transition — safest, no current flowing), OR

- **Motor speed is below 10% of base speed** (low voltage demand, small disturbance from transition), AND
- **No fault flags are active** (healthy system state only)

If a transition is requested while conditions are not met, it is queued and executed automatically when conditions become valid. The user is notified via CAN.

Bumpless Transition Mechanism

1. **Snapshot:** Before transition, the current modulation index (|V_ref| / V_dc) and electrical angle are captured
2. **Crossfade:** Over 10 ms (10 PWM periods at 1 kHz, 160 periods at 16 kHz), duty cycles are linearly interpolated between the outgoing scheme's last output and the incoming scheme's first output
3. **Handover:** At 100% incoming scheme, the function pointer is atomically swapped
4. **Validation:** For 50 ms post-transition, the duty cycle delta between consecutive periods is monitored; excessive jump (>5% of ARR) logs a warning

Transition Matrix

Legal Modulation Scheme Transitions

From \ To	SPWM	SVPWM	SHEPWM	N-Pulse	RSVM	RCFM
SPWM	—	Anytime	Zero torque only	Below 10% speed	Anytime	Anytime
SVPWM	Anytime	—	Zero torque only	Below 10% speed	Anytime	Anytime
SHEPWM	Below 10% speed	Below 10% speed	—	Not allowed	Not allowed	Not allowed
N-Pulse	Below 10% speed	Below 10% speed	Not allowed	—	Not allowed	Not allowed
RSVM	Anytime	Anytime	Zero torque only	Below 10% speed	—	Anytime
RCFM	Anytime	Anytime	Zero torque only	Below 10% speed	Anytime	—

> **Asynchronous to Synchronous Transition**
>
> Transitioning from an asynchronous scheme (SPWM, SVPWM, RSVM, RCFM) to a synchronous scheme (SHEPWM, N-Pulse) requires the PWM frequency to be an integer multiple of the motor electrical frequency. The system automatically adjusts the PWM frequency to the nearest valid synchronous ratio before performing the transition. Transitioning back from synchronous to asynchronous is always allowed (below 10% speed).

6.3 SVPWM (Default at 2 kHz)

Seven-segment symmetric algorithm. Third harmonic injection. Overmodulation Mode 1 and 2 for extended speed range. Default switching frequency: 2 kHz, range 300 Hz - 16 kHz via RTE.

6.4 SHEPWM

Pre-calculated switching angles stored as lookup tables in flash, indexed by modulation index with linear interpolation. V2.0: live on-demand angle calculation (Newton-Raphson) reserved for dynamic operating points.

6.5 N-Pulse Family

1-pulse to 15-pulse synchronous modulation. In synchronous mode, the PWM frequency is phase-locked to an integer multiple of the motor electrical frequency:

N-Pulse Mode vs. Electrical Frequency (at 2 kHz PWM)

Mode	Pulses/Half-Period	Sync Ratio	Electrical Freq Range	Use Case
15-pulse	15	30:1	67 - 133 Hz	Near base speed
9-pulse	9	18:1	111 - 222 Hz	Mid speed range
5-pulse	5	10:1	200 - 400 Hz	High speed
3-pulse	3	6:1	333 - 667 Hz	Very high speed
1-pulse	1	2:1	1000 - 2000 Hz	Overmodulation region

Automatic Mode Transitions

Transitions between N-pulse modes are automatic based on motor electrical frequency with configurable hysteresis to prevent mode chattering:

1. **Upshift** (lower pulse count): Triggered when f_elec rises above the upper threshold of the current mode. Hysteresis: +10% of threshold value.
2. **Downshift** (higher pulse count): Triggered when f_elec falls below the lower threshold. Hysteresis: -10% of threshold value.
3. **Boundary to asynchronous:** When f_elec falls below the minimum for 15-pulse mode, the system transitions to SVPWM (asynchronous). When f_elec rises above the maximum for 1-pulse mode, the system transitions to six-step (full square wave).

Sync/Async Boundary

The boundary between synchronous (N-pulse) and asynchronous (SVPWM/SPWM) operation is user-configurable via RTE. Default: synchronous above 67 Hz elec, asynchronous below. The transition bumplessly adjusts the PWM frequency to the nearest integer multiple of the electrical frequency.

6.6 RSVM / RCFM

LFSR-based pseudo-random sequence seeded from STM32 unique ID. Range configurable via RTE. RSVM randomizes the space vector sequence within each PWM period; RCFM dithers the carrier frequency around a center value. Both modes can be combined (RSVM + RCFM) for maximum EMI spreading.

6.7 PI Controllers

Two PI controllers (d-axis flux, q-axis torque) with anti-windup via back-calculation. Gains stored in FRAM, adjustable via RTE.

6.8 State Machine

Six states with fully defined transitions, guards, and entry/exit actions. Illegal transitions are rejected and logged as faults.

6.8.1 State Transition Table

State Transitions — Guards and Actions

From	To	Guard Condition	Entry Action	Exit Action
INIT	STANDBY	POST complete AND POST passed	—	—
INIT	SAFE	POST complete AND POST failed	Log failure code; SSO	—

STANDBY	READY	Key=ON AND Brake=on AND HV present AND FaultMgr healthy	Close precharge contactor; start precharge timer	—
STANDBY	SAFE	Critical fault detected during STANDBY	SSO; open contactor	—
READY	DRIVING	Key=ON AND Brake=off AND Throttle > deadband AND precharge complete	Enable torque command; start encoder validity timer	—
READY	STANDBY	Key=OFF	Open contactor; zero torque	—
DRIVING	READY	Key=ON AND Brake=on AND Throttle < deadband	Zero torque ref; idle PWM	Disable torque command
DRIVING	FAULT	Non-critical fault active (FAULT_MASK_NONCRITICAL)	Ramp torque to zero (500 ms)	—
DRIVING	SAFE	Critical fault active (FAULT_MASK_CRITICAL) OR Key=OFF	SSO (TIM1_BKIN or PMIC EN kill); open contactor; zero torque	Disable torque command
FAULT	SAFE	Ramp-down complete AND (critical fault active OR timeout 5s)	SSO; open contactor	—
FAULT	READY	All faults cleared AND Key cycled OFF-then-ON AND Brake=on	Run partial POST (sensors, ADC, comms only)	—
SAFE	INIT	Key=OFF-then-ON AND Brake=on AND FaultMgr healthy AND all faults cleared	Run full POST	—

6.8.2 Illegal Transitions

The following transitions are explicitly forbidden and trigger FAULT_STATE_MACHINE if attempted:

- INIT -> DRIVING (POST not yet complete)
- STANDBY -> DRIVING (precharge not yet done)
- READY -> STANDBY while driving (must go through DRIVING -> FAULT/SAFE first)
- DRIVING -> STANDBY (must go through DRIVING -> READY first)
- SAFE -> any state except INIT (requires full POST cycle)
- Any state -> same state with transition attempt (logged as no-op)

6.8.3 State Machine Implementation

```
void SM_Run(void) {
    SystemState_t next_state = sm.current_state;
    uint32_t fault_flags = FaultMgr_GetActiveFlags();

    switch (sm.current_state) {
        case STATE_INIT:
            if (POST_IsComplete() && POST_Passed()) next_state = STATE_STANDBY;
            else if (POST_IsComplete() && !POST_Passed()) next_state = STATE_SAFE;
            break;
        case STATE_STANDBY:
            if (Inputs_KeyOn() && Inputs_BrakeOn() && DCLink_HVPresent()
                && FaultMgr_IsHealthy()) {
                next_state = STATE_READY; Contactor_ClosePrecharge();
            }
            if (fault_flags & FAULT_MASK_CRITICAL) next_state = STATE_SAFE;
            break;
        case STATE_READY:
            if (!Inputs_KeyOn()) { next_state = STATE_STANDBY; Contactor_Open(); }
            else if (Inputs_KeyOn() && !Inputs_BrakeOn()
                     && Throttle_GetPercent() > THROTTLE_DEADBAND
                     && Precharge_Complete())
                next_state = STATE_DRIVING;
            break;
        case STATE_DRIVING:
            if (fault_flags & FAULT_MASK_CRITICAL) {
                SSM_EnterImmediate(); next_state = STATE_SAFE;
            } else if (fault_flags & FAULT_MASK_NONCRITICAL) {
                SSM_RequestRampDown(); next_state = STATE_FAULT;
            } else if (Inputs_KeyOn() && Inputs_BrakeOn()
                       && Throttle_GetPercent() < THROTTLE_DEADBAND) {
                next_state = STATE_READY; TorqueCmd_SetZero();
            } else if (!Inputs_KeyOn()) { SSM_EnterImmediate(); next_state =
STATE_SAFE; }
            break;
        case STATE_FAULT:
            if (fault_flags & FAULT_MASK_CRITICAL) { SSM_EnterImmediate(); next_state
= STATE_SAFE; }
            else if (SSM_RampDownComplete() && FaultMgr_IsHealthy()
                     && sm.fault_time + 5000 < HAL_GetTick())
                next_state = STATE_READY;
            break;
        case STATE_SAFE:
            if (!Inputs_KeyOn()) sm.reset_request = true;
            if (sm.reset_request && Inputs_KeyOn()
                && Inputs_BrakeOn() && FaultMgr_IsHealthy()) {
                next_state = STATE_INIT; POST_Start();
            }
            break;
    }

    /* Validate transition legality */
    if (next_state != sm.current_state) {
        if (!SM_IsTransitionLegal(sm.current_state, next_state)) {
            FaultMgr_SetFault(FAULT_STATE_MACHINE, (sm.current_state << 8) |
```

```
next_state);
            next_state = STATE_SAFE; /* forced safe on illegal transition */
        }
        SM_ExitState(sm.current_state);
        sm.current_state = next_state;
        SM_EnterState(sm.current_state);
    }
}
```

6.9 Torque Commander

Converts throttle to I_q reference. Rate limits: ramp-up 500 A/s, ramp-down 2000 A/s, regen 1000 A/s. Four torque maps + Sport/Eco. All stored in FRAM via RTE.

6.10 Real-Time Loss Estimator and Thermal Model

The system continuously estimates the power dissipated in the IGBTs using a pre-programmed loss model. This model combines conduction losses, switching losses, and a thermal RC network to estimate junction temperature and predict thermal transients.

6.10.1 Loss Model

Power Loss Components

Loss Type	Formula	Parameters (User-Configurable)
Conduction	P_cond = V_ce(sat) x I_c x D, where V_ce(sat) = V_ce0 + R_ce x I_c	V_ce0 (threshold, ~0.8V), R_ce (slope resistance, ~3 mOhm), from IGBT datasheet
Switching (turn-on)	P_sw_on = E_on x f_sw, where E_on from datasheet energy curve at operating V_dc and I_c	E_on vs (V_dc, I_c) lookup table from datasheet; gate resistance R_g
Switching (turn-off)	P_sw_off = E_off x f_sw	E_off vs (V_dc, I_c) lookup table; R_g
Diode (FWD)	P_diode = V_f x I_f x (1-D) + E_rec x f_sw	V_f, R_diode from datasheet; E_rec vs I_c
Total per IGBT	P_total = P_cond + P_sw_on + P_sw_off + P_diode	All stored in FRAM, editable via RTE per module type

6.10.2 Thermal Model

A lumped-parameter thermal RC network models heat flow from junction to ambient:

$$T_{junction} = T_{sensor} + P_{total} \times (R_{th\_jc} + R_{th\_cs} + R_{th\_sb}) \times (1 - e^{(-t/\tau)})$$

Where T_sensor is the measured NTC temperature, R_th_jc is junction-to-case thermal resistance, R_th_cs is case-to-sink (TIM), R_th_sb is sink-to-baseplate, and tau is the thermal time constant of the sensor mounting location. All thermal parameters are user-configurable via RTE for different IGBT modules and cooling systems.

6.10.3 Estimated Junction Temperature

The model computes an estimated junction temperature every 1 ms. This estimate is compared against the measured sensor temperature:

- If T_junction_est > T_sensor_measured + 20C for > 5 seconds: flag FAULT_THERMAL_MISMATCH — the sensor may be detached or the thermal path degraded

- If T_junction_est > 125C: early warning — aggressive derating before the 100C sensor hard cap is reached

6.10.4 Time-to-Temperature Prediction

Using the thermal model, the system predicts how long until the sensor temperature reaches each threshold:

```
/* LossEstimator.c — 1 ms tick */
void LossEstimator_Update(void) {
    /* 1. Calculate instantaneous power per IGBT */
    float i_phase = fabsf(adc_filtered[phase_index]); /* RMS approx */
    float duty = fabsf(duty_cycle[phase_index]);
    float v_sat = igbt_params.V_ce0 + igbt_params.R_ce * i_phase;
    float p_cond = v_sat * i_phase * duty;
    float p_sw = (igbt_params.E_on + igbt_params.E_off) * pwm_frequency;
    float p_total = p_cond + p_sw;

    /* 2. Update thermal model (discrete RC) */
    float t_ambient = temp_ambient; /* from sensor or estimated */
    float tau = thermal_params.tau_sensor; /* thermal time constant */
    float r_th = thermal_params.R_th_total;
    float dt = 0.001f; /* 1 ms */
    t_junction_est += (dt / tau) * ((t_ambient + p_total * r_th) - t_junction_est);

    /* 3. Predict time to thresholds */
    if (p_total > 0.0f) {
        float delta_to_warn = 95.0f - t_sensor_measured;
        float delta_to_fault = 100.0f - t_sensor_measured;
        float heating_rate = p_total * r_th / tau; /* C/s approximation */
        if (heating_rate > 0.1f) {
            time_to_warn_sec = delta_to_warn / heating_rate;
            time_to_fault_sec = delta_to_fault / heating_rate;
        }
    }

    /* 4. Publish to CAN/RTE */
    telemetry.p_junction_est = t_junction_est;
    telemetry.p_total_loss = p_total * 6.0f; /* all 6 IGBTs */
    telemetry.time_to_ot_sec = time_to_fault_sec;
}
```

The time-to-overtemperature prediction is published on CAN and visible in RTE. This gives the rider advance warning: "at current power, IGBT will reach thermal limit in 45 seconds." The system also uses this internally to pre-emptively reduce torque before the hard cap is reached (predictive derating).

7. Application Layer

Non-safety-critical modules providing vehicle-level functionality. Target: ASIL-B. All persistent data stored in SPI FRAM with write protection.

7.1 Throttle Processing

Dual potentiometer processing: simultaneous ADC read, range check (0.5V - 4.5V), plausibility check (<5% delta), map to 0-100%, deadband (0-5%, user-configurable), IIR filter. Failure: FAULT_THROTTLE_PLAUS, zero torque, FAULT state.

7.2 CAN Communication

User-tailorable protocol. Reference frames in Section 10 are defaults only. RX dispatch via ID-based callback table. RTE commands handled on dedicated CAN ID.

7.3 DC-Link Monitor

16-bit ADC, 10 ms IIR filter. Thresholds user-configurable via RTE within hard bounds (OV max = hardware rating, UV min = 60V). Cross-checked against back-calculation formula.

7.4 Temperature Monitor (1oo3)

1-out-of-3 voting: highest reporting sensor wins. No averaging, no median. If one sensor reports 95C and the others report 70C, the system uses 95C.

> **Stuck-High Sensor Vulnerability**
>
> Because "highest wins" has no outvoting mechanism, a single noisy or stuck-high sensor can trigger progressive derating (from 80C) or fault entry (at 100C) even when the other two sensors report normal temperatures. The sensor validity check (Section 7.4.2) only catches readings outside the physical range (-40C to +175C). A sensor stuck at 95C (within range) causes unnecessary derating. There is no 2-out-of-3 consensus mechanism. This is a known limitation: the design prioritizes thermal safety (never miss a real overheat) over availability (may derate on a false positive).

IGBT Temperature Thresholds

Level	Temp	Response
Normal	< 80 C	Full torque

Derate 1	80 C	Linear derate: 100% to 50% torque (80-90C range)
Derate 2	90 C	Torque limited to 25%
Warning	95 C	FAULT_IGBT_OT_WARN logged
Fault	100 C	FAULT_IGBT_OT, ramp to zero, FAULT state

Each sensor checked against valid range (-40C to +175C). Two or more invalid sensors: FAULT_SENSOR_FAIL, FAULT state.

7.5 POST (Power-On Self Test)

POST runs once after every reset using X-CUBE-CLASSB (IEC 60730-1 Class B) for CPU register test, RAM March-C, and flash CRC. Additional application-specific tests:

POST Test Checklist

Step	Test	Pass Criteria
1-4	X-CUBE-CLASSB: CPU registers, RAM walking-bit, RAM March-C, flash CRC	All Class B tests pass
5	ADC self-test (VREFINT, TSENSE)	Within expected range
6	Gate driver FAULT pins	All de-asserted (high)
7	Encoder signal presence	Valid sin/cos if rotating
8	CAN loopback	Frame received
9	Watchdog self-test (force timeout, verify reset cause)	Reset cause = IWDG
10	Current sensor VREF check	All REF within 2.48-2.52V window
11	Current sensor zero-point check	All within +/-20% of zero
12	FRAM read/write test	Pattern read back correctly
13	FRAM write-protection pin test	WP pin toggles correctly

7.6 RTE Configuration Interface

The **Real Time Examiner (RTE)** is an open-source host tool connecting via CAN bus. It provides live monitoring and runtime parameter configuration. Parameters stored in SPI FRAM (not flash) with unlimited write endurance.

RTE-Configurable Parameters

Category	Parameters
Motor Control	Modulation scheme, PWM frequency (300 Hz - 16 kHz), PI gains, current limits, field weakening
Torque Maps	4 custom 16-point curves + Sport/Eco presets, ramp rates, deadband
Safety Thresholds	DC-link OV/UV, IGBT temp levels, throttle plausibility delta, CAN timeouts
Loss Model	V_ce0, R_ce, E_on/E_off curves, R_th values, tau — all per IGBT module type
Vehicle Data	Odometer (km), hour counter, boot counter, fault log (256 entries)
CAN Protocol	All frame IDs, scaling factors, periods — fully user-defined
Custom	32 float + 32 uint32_t user parameters

Changes validated before application (Section 7.7). Committed to FRAM only on explicit "save" command. FRAM WP pin asserted during normal operation, de-asserted only during commit.

7.7 Input Validation Framework

Every user-configurable parameter passed via RTE is validated before being applied. The validation framework has three layers:

7.7.1 Layer 1: Hard Bounds (Compile-Time)

Safety-critical parameters have absolute min/max values compiled into the firmware. These **cannot be overridden** by RTE or any other interface. They represent the physical limits of the hardware.

Hard Bounds Examples

Parameter	Hard Min	Hard Max	Rationale
IGBT temp fault threshold	60 C	100 C	100 C hard cap (75 C margin to Tj_max 175 C)
DC-link OV threshold	200 V	400 V	400 V = hardware voltage rating limit
DC-link UV threshold	60 V	150 V	Below 60 V: ADC quantization dominates
PWM frequency	300 Hz	16 kHz	Below 300 Hz: excessive ripple/noise; above 16 kHz: thermal
Dead time	500 ns	4 us	Below 500 ns: shoot-through risk; above 4 us: excessive loss
Max torque (I_q_ref)	0 A	350 A	350 A = hardware current limit with margin
Current sensor VREF min	2.40 V	—	Below 2.40 V: sensor fault or supply collapse

7.7.2 Layer 2: Cross-Parameter Consistency

Parameters are validated against each other for logical consistency:

- DC-link UV threshold must be < DC-link OV threshold (minimum 10 V separation)
- Temperature derate start must be < temperature fault threshold (minimum 10 C separation)
- Throttle deadband must be < 20%
- PI gains must be positive (negative gain = positive feedback = runaway)
- Modulation scheme must be valid enum value (0-7)
- Switching frequency must be compatible with selected modulation (SHEPWM has minimum frequency)

7.7.3 Layer 3: Rate-of-Change Limiting

To prevent abrupt changes that could destabilize the control loop:

- PI gain changes: maximum 10% per RTE commit
- Switching frequency changes: only at zero torque
- Modulation scheme changes: only at zero torque or below 10% base speed
- Safety threshold changes: only in STANDBY or READY state (not while driving)

7.7.4 Validation Framework Implementation

```
/* InputValidator.c — three-layer validation */
ValidationResult_t InputValidator_Check(const ParamDef_t* def, float value) {
    /* Layer 1: Hard bounds */
    if (value < def->hard_min || value > def->hard_max) {
        return VALIDATION_HARD_BOUND_VIOLATION;
    }
    /* Layer 2: Cross-parameter consistency */
    if (def->consistency_check != NULL) {
        if (!def->consistency_check(value)) {
            return VALIDATION_INCONSISTENT;
        }
    }
    /* Layer 3: Rate of change */
    float delta = fabsf(value - def->current_value);
    if (delta > def->max_delta) {
        return VALIDATION_RATE_LIMITED;
    }
    return VALIDATION_OK;
}

/* Example: IGBT temp threshold validation */
bool Validator_TempFaultConsistent(float new_threshold) {
    /* Fault threshold must be > derate start + 10C */
    return (new_threshold > params.igbt_derate_start + 10.0f);
}
```

8. Data Flow and Interfaces

Module-to-Module Data Interfaces

Source	Data	Consumer(s)	Freq	Method
ADC oversampler	I_U, I_V, I_W (decimated)	FOC, LossEstimator	PWM freq	DMA double buffer
ADC oversampler	I_U, I_V, I_W (raw samples)	Ripple monitor	n x PWM freq	DMA circular
Encoder	theta_e	FOC	PWM freq	Function call
Throttle	Throttle %	TorqueCmd	1 kHz	Function call
TorqueCmd	I_q ref	FOC	100 Hz	Atomic float
DCLink Mon	V_DC	FOC, CAN	100 Hz	Function call
Temp Mon	T_IGBT (highest)	FaultMgr, LossEstimator	10 Hz	Function call
LossEstimator	T_junction_est, P_loss, time_to_OT	CAN, RTE, FaultMgr	1 kHz	Global struct
FaultMgr	Fault flags	SM, CAN, RTE	On change	Volatile uint32
State Machine	Current state	FOC, TorqueCmd, CAN	100 Hz	Global enum
CAN RX	BMS/IO/charger data	Handlers	Event	Callback dispatch
RTE CAN	Param read/write	RTE handler	Event	FRAM access

> **Cache Coherency**
>
> DMA buffers for ADC results placed in SRAM4 (non-cacheable via MPU). All DMA buffers use cache-line-aligned addresses. The FOC ISR reads ADC DMA results — cache invalidation is not required because the DMA buffer resides in non-cacheable SRAM4.

9. Memory Layout

9.1 Flash Memory Map

Flash Memory Map

Region	Address	Size	Content
Sector 0	0x0800_0000	8 KB	Bootloader
Sector 1	0x0800_2000	8 KB	Bootloader config (HMAC key)
Sector 2	0x0800_4000	8 KB	Reserved
Sectors 3-7	0x0800_6000	40 KB	Reserved
Bank 1 App	0x0801_0000	448 KB	Application firmware
Bank 2 Swap	0x0808_0000	512 KB	OTA update swap

No RDP (open-source). No OTP. WRP on bootloader sectors only.

9.2 RAM Memory Map

RAM Memory Map

Region	Size	Usage	ECC
AXISRAM	512 KB	Stack, heap, FOC buffers, fault log runtime	Yes (SECDED)
SRAM1	128 KB	DMA buffers, CAN FIFOs (non-cacheable)	No
SRAM2	128 KB	ADC oversample buffers, DMA descriptors	No
SRAM4	64 KB	Warm-boot retention (fault, cycle count)	No

> **ECC Handling (FSR-20)**
>
> AXISRAM has SECDED ECC. Single-bit errors corrected by hardware. Double-bit errors trigger NMI: log address, set FAULT_ECC_RAM, trigger safe state, avoid re-reading corrupted address.

10. CAN Protocol

Reference implementation — user-tailorable via RTE. All IDs, scaling factors, and periods are defaults stored in FRAM and modifiable.

VCU TX Frame Reference (Defaults)

CAN ID	Name	Period	Key Signals
0x18F00100	VCU_Status	100 ms	State, fault flags
0x18F00200	VCU_Motor	50 ms	Speed (rpm), torque (Nm), I_q, I_d
0x18F00300	VCU_Power	100 ms	V_DC, I_DC, power (kW)
0x18F00400	VCU_Thermal	500 ms	T_IGBT[0-2], T_junction_est
0x18F00500	VCU_LossEst	500 ms	P_total_loss, time_to_OT_sec
0x18F00600	VCU_Heartbeat	1000 ms	Seq counter, uptime
0x18F00700	VCU_FaultLog	Event	Event code + detail

11. Traceability Matrix

FSR-to-Module Traceability

FSR	Description	ASIL	Module(s)
FSR-01	MCU clock/supply monitoring	C	PSU_Monitor, POST
FSR-02	Key ON before HV	C	State Machine
FSR-03	Safe state on fault	C	SSM, FaultMgr, TIM1_BKIN, RailSwitch
FSR-04	Dead time, no shoot-through	C	TIM1 DTG
FSR-05	Overcurrent shutdown	C	ADC comp + FOC ISR
FSR-06	Redundant current measurement	C	ADC multi-rate, CurrentSensor_Validate
FSR-07	Throttle plausibility	C	Throttle module
FSR-08	Torque rate limiting	B	TorqueCmd ramp limiter
FSR-09	DC-link OV/UV	C	DCLink_Monitor
FSR-10	Precharge timing	B	State Machine
FSR-11	IGBT over-temperature	C	TempMonitor (1oo3), LossEstimator
FSR-12	Contactor weld detect	B	Contactor driver
FSR-13	Key-off torque zero	C	State Machine
FSR-14	Direction interlock	B	State Machine
FSR-15	Encoder validity	C	Encoder, ENC_Monitor
FSR-16	Brake-torque interlock	B	State Machine
FSR-17	CAN heartbeat	B	CAN heartbeat check
FSR-18	Watchdog	C	WatchdogMgr (IWDG+WWDG+ext)
FSR-19	Boot CRC	C	Bootloader
FSR-20	RAM ECC	C	NMI handler, FaultMgr
FSR-21	POST before drive	C	POST, State Machine

12. Bootloader Architecture

Minimal second-stage bootloader in flash Sector 0. Runs on every reset before the application.

```
void Bootloader_Main(void) {
    SystemClock_Config_Basic();
    ResetReason_t reason = Boot_GetResetReason();

    /* Boot CRC check (FSR-19) */
    uint32_t app_crc = Boot_CalculateCRC32(APP_START_ADDR, APP_SIZE);
    uint32_t stored_crc = *(uint32_t*)APP_CRC_ADDR;
    if (app_crc != stored_crc) { Boot_EnterUpdateMode(); return; }

    /* Check for update request (500 ms window) */
    if (reason == RESET_POWER_ON || reason == RESET_PIN) {
        FDCAN_Init_Basic();
        if (Boot_CheckForUpdateRequest(500)) { Boot_EnterUpdateMode(); return; }
        UART_Init_Basic();
        if (Boot_CheckForUSBUpdateRequest(500)) { Boot_EnterUpdateMode(); return; }
    }
    Boot_JumpToApplication(APP_START_ADDR);
}
```

13. Firmware Update Protocol

Firmware updates accepted over **UART/USB** or **CAN** — both use the same protocol. The update interface is selected automatically during the 500 ms bootloader window (CAN frame or USB enumeration detected).

13.1 Update Protocol

Firmware Update State Machine

State	Description	Transitions
IDLE	Waiting for update request	RX request -> AUTH
AUTH	HMAC verify request, check rollback counter	OK -> RX; Fail -> IDLE
RX	Receiving chunks, write to Bank 2	All chunks -> VERIFY; Timeout -> IDLE
VERIFY	SHA-256 full image + HMAC signature	OK -> COMMIT; Fail -> IDLE
COMMIT	Atomic bank swap, update rollback counter	OK -> REBOOT
REBOOT	System reset	-> Bootloader

13.2 Security Requirements

- Update only when stationary (CSR-04: speed=0, brake=on, key-on)
- Chunks: sequence number + CRC-32; missing chunks re-requested
- Complete image SHA-256 before flash commit
- HMAC-SHA256 signature with user-managed key from bootloader config
- Anti-rollback counter (CSR-03); user-resettable via JTAG
- Dual-bank atomic swap

13.3 FRAM Protection During Update

During firmware update, the FRAM write-protection (WP) pin is **asserted** (high) to prevent any accidental parameter corruption. The bootloader does not access FRAM. After a successful update, the application verifies FRAM integrity on first boot and restores from a shadow copy in flash if corruption is detected.

> **Open-Source Trust Model**
>
> No OTP fuses. No vendor-locked keys. HMAC key stored in plaintext bootloader config. Physical access allows key modification via JTAG. User is root of trust.

14. Scheduling and Timing

14.1 Interrupt Priority Map

NVIC Priority Assignment

Prio	IRQ	Source	Max Latency
0	TIM1_UP	FOC loop + PWM update	0 us (no preemption)
1	ADC_DMA	ADC oversample buffer full	5 us
2	TIM1_BRK	Gate driver fault (SSO)	<10 us (hw handles)
3	TIM6	1 ms system tick	100 us
4	FDCAN1/2_RX0	CAN receive	200 us
5	TIM7	Watchdog service	10 us
6	FDCAN_TX	CAN TX complete	N/A
7	USART	Debug / USB	N/A

14.2 1 ms System Tick (TIM6)

```
void TIM6_DAC_IRQHandler(void) {
    HAL_TIM_IRQHandler(&htim6);
    SafetyMonitor_ADCCheck();
    Throttle_Process();
    DCLink_Monitor();
    ENC_Monitor();
    LossEstimator_Update();          /* junction temp + time-to-OT */
    if ((tick_count % 50) == 0) { WWDG_Refresh(); IWDG_Refresh();
ExternalWDT_Toggle(); }
    if ((tick_count % 10) == 0) { SM_Run(); TorqueCmd_Update(); }
    if ((tick_count % 100) == 0) { CAN_TX_Heartbeat(); CAN_CheckHeartbeats(); }
    if ((tick_count % 100) == 0) { Temp_Monitor_1oo3(); CurrentSensor_Validate(); }
    tick_count++;
}
```

14.3 WCET Summary

Worst-Case Execution Time

Task	Period	WCET	Util

FOC ISR (TIM1_UP)	62.5 us - 3.3 ms	5.0 us (SHE worst case)	8.0% @ 16 kHz / 0.15% @ 300 Hz
ADC DMA callback	20.8 us	0.5 us	2.4%
TIM1 break ISR	Event	0.3 us	Negligible
TIM6 tick	1 ms	60 us (incl. LossEstimator)	6.0%
CAN RX ISR	Event	10 us	<1%
Total @ 16 kHz PWM (worst case)			**<18%**
Total @ 2 kHz PWM (nominal)			**<9%**

15. Assumptions and Limitations

15.1 Assumptions

ID	Assumption
A-01	Gate driver FAULT outputs OR'd to TIM1_BKIN
A-02	16-bit ADC1 primary + 12-bit ADC2/ADC3 redundant
A-03	Motor parameters (Ld, Lq, lambda_m, poles) known; stored in FRAM via RTE
A-04	3.3V supply stable within +/- 10%
A-05	Physical tampering out of scope (open-source trust model)
A-06	Single-core, no RTOS
A-07	Single CAN bus (no redundancy)
A-08	User has STM32CubeMX for GPIO and peripheral configuration
A-09	Loss model parameters (V_ce0, R_ce, E_on, E_off, R_th) entered via RTE for user's specific IGBT module
A-10	FRAM chip present and functional (verified by POST test 12)

15.2 Known Limitations

ID	Limitation	Mitigation
L-01	Single-core MCU, no lockstep	External watchdog; POST; 3-path SSO
L-02	SHEPWM: pre-computed tables only (no live calc in V1.0)	Tables cover typical range; V2.0 reserved
L-03	Single CAN bus	Safe state on timeout
L-04	No field weakening in V1.0	Planned V2.0
L-05	Gate drivers AEC-Q100, not ASIL-rated	DESAT+UVLO+dead time; ASIL C max
L-06	HMAC key in plaintext flash	Intentional; user is root

L-07	No regen current limit in V1.0	BMS backstop; V1.1
L-08	STL is Class B (X-CUBE-CLASSB), not Class D	Documented in HARA LIMIT-07; 71 fault injection tests
L-09	Loss model accuracy depends on user-entered IGBT parameters	Defaults from datasheet; user tunes via RTE; thermal model cross-checks against sensor
L-10	No Dependent Failure Analysis (DFA) per ISO 26262-9	Acknowledged in HARA. Common-cause between SSO paths (e.g., PCB delamination affecting both TIM1_BKIN and PMIC EN routing) not formally analyzed. Required for ASIL C claim.
L-11	No EMI/EMC pre-compliance assessment	Environmental EMI tests defined (E-08, E-09) but no pre-compliance testing or design margin analysis completed. Risk in unknown vehicle installations.
L-12	12-bit ADC redundancy has coarser resolution than 16-bit primary	The 5.3x resolution difference is not a safety issue for fault detection (broken wire, stuck output are reliably caught). Subtle gain drift is not safety-critical. A second 16-bit ADC would provide higher-fidelity redundancy but is not required for ASIL-C.
L-13	1oo3 temp voting: single stuck-high sensor can cause unnecessary derating	No 2-out-of-3 consensus mechanism. A sensor stuck at 95C (within valid range) triggers derating even if the other two read 70C. Trade-off: thermal safety over availability.

15.3 Future Work (V2.0+)

- Live on-demand SHE angle calculation (Newton-Raphson/homotopy)
- Field weakening for extended speed range
- Sensorless FOC as encoder backup
- CAN FD at 2 Mbps
- Auto-tuning of PI gains
- Bluetooth LE for wireless diagnostics
- SD card data logging